

Lec 2: Text Classification with Linear Models | 主题: Text classification 任务 → BoW → baseline (kNN) → Linear classifier

- Text Classification: Binary, Multi-Class (SoftMax, 多类单选, choose one of K classes), Multi-Label (Sigmoid)
- Input representation: 把 text 转成 feature vector ($x = f(\text{text})$)
- Bag-of-Words (BoW) 定义: 忽略语序/语法、顺序, 排列。BoW 的问题: 高维稀疏 sparsity、loss of sentence structure (word order) BoW 的维度是整个 vocabulary 的大小 $|V|$
- 第一种分类器 classifier: Nearest Neighbor / kNN baseline。 距离度量示例: L1 distance, 词越相近, 计数越相近, 距离越小, $k = 1$ 高反差过拟合, k 大平滑方差小 good
- 第二种分类器 Linear classifier: $f(x, W) = Wx + b$ | trainset 是学参数, validset 是选超参数, test 是跑结果看泛化

Lec 3: Linear Models: Loss Function | 主题: Linear Classifier → SoftMax → Loss Function → MLE → 正则化

- Softmax classifier: scores → probabilities. 把 score 转成条件概率 $P(Y=k|X)$ 。线性分类器的 score: $s = Wx$ (unnormalized scores)。
- SoftMax 是 mult-class 的, sigmoid 是 binary 的, 但是可以把 SoftMax 简化成 binary。Score 大且为正—class=1; score 大且负, =0
- Loss function: 从“给正确类多大概率”到可优化目标。 直觉: 正确类概率高 → loss 低; 正确类概率低 → loss 高。 最大化正确类概率=最大化 $\log P$ =最大化 p =最小化 loss。 Softmax 越均匀, loss 越小
- Maximum Likelihood Estimation(MLE) 最大似然估计 (定义了 data loss 的形式): 目标是选 W 让观测到的标签咋模型下尽可能可能。 通常优化会改成: 最小化负对数似然 NLL, 在 softmax 里就是交叉熵 loss, 所以 data loss = 负 log-likelihood, MLE 等于最小化 NLL 因为我们的目标是让 model 在未见的数据上也能表现的最好, 但是光靠 MLE 会过拟合, 所以我们要正则化 Regularization
- 在总损失中 L_2 为正则化, 有 λ , 它=0 则无正则, $=\infty$ 则只关心正则, 我们希望最好=0.5 左右以平衡

Lec 4: Linear Models: Optimization | 主题: softmax + cross-entropy → 优化目标是最小化 $L(W)$ → 数值梯度/解析梯度 → 梯度下降法 SGD

- Optimization intuition: 优化问题实质上是多维中的梯度下降问题, 梯度指向函数增长最快的方向, 要降低 loss, 应沿负梯度方向移动 / 梯度下降的目标是 minimize loss function $L(W)$ 。loss 要小, $L(W)$ 也要小=好
- Gradients
 - 数值梯度 Numerical Gradients: 不足时 require looping over every parameter, 对多权重的 models 不灵活
 - 解析梯度 Analytic Gradients: 针对 softmax loss 推导梯度, 算 $\text{deltaw}L(W)$, 数值梯度 slow & approximate, 解析梯度 fast & exact。 (NG 法对参数扰动频繁, 参数多时不可用, AG 一次反向传播, 所以梯度都会更新, 因此好)
- Gradient descent / SGD 梯度下降法(属于优化器, 其他还有 Adam): 从一开始的参数 W 开始计算梯度 $\text{deltaw}L(W)$, 每进行梯度下降更新参数直到 loss 收敛。反复计算梯度并更新参数以减少损失。(梯度下降的本质解释: 在参数空间里, 把 loss 当地形; 梯度给出“上升最快方向”, 所以取负梯度就是“下降最快方向”, 每次沿这个方向走一小步以降低 loss。)
- Backpropagation 反向传播算法: 用于计算损失函数相对于所有模型参数的梯度。基于微积分链式法则+复用中间计算结果
- Linear model 与 classification 任务的回顾: 把 text 代表成向量, 向量 score 写成线性模型, 用 softmax 把分数变成概率, 然后选择最高的概率, 使用正则化等手段训练。Linear Classifiers 好处是 simple, fast, classes 线性时 separable, 但是 model 时复杂/non-linear 时候就很差, 所以需要前馈神经网络 FFNN 来加入 nonlinearity

Lec 5: FeedForward Neural Networks | 主题: FFNN → activation/nonlinearity → 为什么 non-linear, stacking → 训练 (regularization) → DAN

- FFNN: 没有循环结构的神经网络。输入 vector → 多层线性变换 → 激活函数 → 输出。FFNN 可以学 non-linear 但输入长度必须固定。多层神经网络每层都重代表线性映射, 最后一层 classifier 一定是 non-linear。FFNN 的输入是一个固定维度的向量。
- Activation function 激活函数 (非线性): 神经网络要有激活函数, 常用为 ReLU, 还有 sigmoid/tanh/ELU/Maxout/GeLU
- 为什么激活函数 g 一定是非线性的: 要让 data 变得 linearly separable; 如果 linear 则多层叠加 stacking layer 不会增加模型的表达能力, 线性复合也为线性, 不好。激活函数主要在 hidden layer 起作用
- 为什么正则化 Regularization 在神经网络里重要: $L_2 = \lambda R(W_2)$, 提高 λ 可以以 model complexity 换取 generalization 泛化能力。正则化方法: Dropout, 早停, L_2 weight decay, Data augmentation
- Deep Averaging Networks (DAN, 是 FFNN 之一): 把 input text 的 word embedding 求平均。Embedding layer + averaging + FFNN + softmax

Lec 6: Learning Word Representations | 主题: word embeddings → Word2Vec/Skip-gram → GloVe → tokenization → subword-level & OOV

- Word embeddings: 每个词可以被 vectors of real numbers 表示, 相近的词有相似的 vector
- Word2Vec (一系列简单高效的词嵌入学习模型), 其中最主要的一种办法是 Skip-gram, 目标: 给定中心词预测窗口内上下文词 (窗口是固定的)。训练 skip-gram 的损失函数是用 linear model + softmax, 会用到 center word 和 context word。是无监督学习!
- 如何评估 word embeddings 是好是坏, 使用预训练的 word embeddings 初始化, 然后在任务上 finetuning 看效果。Word2Vec 比较低效, 它要遍历整个语料库中的每个词, 预测每个窗口的中心词, 一次只能捕捉一个词的 cooccurrence
- GloVe: 一次性捕捉共生关系。全局共现计数矩阵 Global cooccurrence counting matrix
- Tokenization: 是将原始文本转换为一系列 tokens 的过程。Tokenizer 是一个独立的模块, 是 text 和 token 化的桥梁
- Word-level tokenization 存在问题, 可能会 out-of-vocabulary (OOV) 和形态爆炸问题。Character-level 可以解决这个问题但是 less expressive than words 并且序列长, 折中方案是 subword-level: vocabulary 适中, 可以通过组合已知子词来表示未见过的单词。

Lec 7: Language Modeling + RNNs | 主题: LM perplexity → RNN → explode/vanishing gradients → LSTM → RNN/FFNN → n-gram

- Language modeling: 学习哪些词序列更可能出现, 并把整句概率分解成逐词条件概率的乘积。衡量 LM 质量的指标是 perplexity 困惑度, LM 分配概率给可能的下一个词, 困惑度就是衡量 model 的选择有多么不确定 uncertain, 困惑度是指模型做出的有效选择数量。
- Recurrent neural network RNN LMs: 传统的固定窗口输入 (FFNN) 的 LM 因为窗口太小, w 会不停的过大; 所以 RNN 通过 hidden states 来处理可变长度序列, 有 variable-length context 窗口, 在每个时间点重复使用同一组权重 W 做到参数共享。
- Vanishing/explode gradients: 因为 RNN 随时间展开, 所以导致一个非常长的乘法链条在反向传播里, 会导致梯度爆炸或者消失。梯度爆炸是反向传播后梯度变得非常大, 会导致训练不稳定, 发散, 震荡 oscillate, 处理办法是 gradient clipping; 梯度消失是反向传播后梯度接近 0, 模型没有学到东西, 参数更新慢, 记不住很久之前的消息
- 怎么去解决 vanishing gradients: LSTM: 能更好的处理长序列, 在每个 time-step 维护一个 cell state, 有 gated memory 结构, 用于控制循环神经网络中的记忆和遗忘过程 (cell state + gates + long-range dependencies + mitigate vanishing gradients)。更能学到长距离依赖

- RNN 是通用的 Sequence Encoders，可以用于表示文本序列（语言建模，文本分类，Sequence labeling），好处：可处理任何长度，model size 不会随着 longer input 增加，计算可复用；缺点是计算忙，事实上很难复用，会有记忆衰减
- FFNN 与 RNN 的区别：在 FFNN 中每一层有自己独立的参数，但是 RNN 共享每一层参数；FFNN 是固定维度输入窗口，RNN 是动态，FFNN 没有跨时间样态，没有记忆，RNN 有 hidden states，同一时间维度参数共享；RNN 能建模顺序，有上下文依赖，FFNN 无
- n-gram：对比 RNN 的一种基线，只看只看固定窗口的前 n-1 词来预测下一个词。快但上下文固定且 sparsity，RNN 任意长度。

Lec 8: Machine Translation + Seq2Seq | 主题：MT/seq2seq → Vanilla→seq2seq with attention→ beam search/Greedy decoding/ Exhaustive search

- Machine Translation：要求语言的分析和生成能力，因此有 NMT 神经机器翻译，它是一个端到端的神经网络架构叫做 seq2seq 包含两条 RNNs（Encoder 和 Decoder）。如何训练 NMT/Seq2seq 模型呢：监督学习，先把 source language word embeddings 查到，然后用 RNN/LSTM 逐词读入，得到 encoder hidden vector，其中最后一个 hidden vector encoding 总结了 source sentence。然后在 Decoder 这里，也是从 target language word embeddings 开始，到 decoder hidden vectors，从看到 START 开始到看到 END 结束。
- 传统的 Vanilla Sequence-to-Sequence 只看 source sentence 的 summary 在翻译时候，但是带有 attention 的 Seq2Seq 可以在翻译时候是不是回溯源句子，并且会 align output 与 input。具体来说是会计算相似度分数在 each time step 的 decoder hidden vector 和 encoder hidden vector，然后将相似度得分向量通过 SoftMax 转化成 attention weights 最后求和生成 context vector，再之后就在 decoder 和之前一样
- Attention mechanism：注意力机制是深度学习中的一种重要手段，一般来说是有 values 和 query，这个过程一般包括计算 attention scores，用 SoftMax 算 attention distribution，然后求 weighted sum of values，最后就能得到 attention outputs 也叫 context vector
- Greedy decoding：逐步选当前概率最大的词/token，快速确定性的输出，但是容易产生局部最优，生成质量差，无法回溯，风险大
- Exhaustive search decoding：计算 all possible sequences，太 expensive 太耗时，不可能
- Beam search decoding（折中最现实）：一步只保留 top-k 个 hypotheses。k 是 beam size，常用 5-10。每个 hypothesis 的 score 用 log probability（log 概率相加）：分数都为负，但 越大（越接近 0）越好。每步拓展后选总分最高的 k 条。k=1 就是 greedy
- NMT less interpretable, hard to debug, and difficult to control

Lec 9: Self-Attention & Transformers | 主题：self-attention→ Q/K/V 形式化 → scaled dot-product→ Multi-head attention→ encoder block

- 如何解决梯度爆炸和忘记 far-back info → Self-attention: transforms context-free representations into context-sensitive ones.其形式化是 queries,keys 和 values。首先给到 query vector q，和 key-value(k,v)，计算 q 和 k 的相似度权重 a，然后输出 weight sum h = 累加求和 a v
- 注意力机制 Attention mechanism = 1. $\text{Score} = q \cdot k$ ； 2. 通过 SoftMax 正则化得到 a； 3. 通过加权和 aggregate 形成 $A(q, K, V)$ 。
- a. 在上一节的 seq2seq 的 cross-attention 范式汇总，values 来自 source sentence，queries 来自 target sentences；但是在 self-attention 中，queries, keys 和 values 都来自于 same sequence。
- b. RNN 是每个 hidden state 依赖上一个，是串行 inherently sequential；但是自注意力是 parallel 的并且每个 output 依赖于 all inputs
- 为什么 scaling 在 SoftMax 里重要：attention 权重来自 softmax，但 softmax 梯度依赖于输入尺度，点积 dot production 过大时候 SoftMax 接近 one-hot，梯度几乎为 0。办法就是 scale dot product by: 除去 scale 点积 attention，替代有 additive, multiplicative
- Multi-head attention：单头瓶颈：不同 relevance 需求无法同一分布表达；multi-head 却可以保持注意力机制前提下每个 head 有自己独立的 Q/K/V 投影，把同一输入映射到 different representation subspaces；多个 head 并行学不同 relevance。输出还是 Z 但更 expressive
- Self-Attention in the Transformer Encoder：多头自注意力机制 can model diff type of interactions between inputs tokens，但它是 linear 的，输出值是加权和，no non-linear transformations across features。为表达 non-linear 的力量需要 Transformer
- 接上，在其中有 feedforward layer（encoder）：是 position-wise 计算，attention 是 tokens 交换信息，feedforward 是 tokens 在内部 transform 信息，单个模块以及可以表达上下文 contextualized representation，但是需要多层叠加，叠加每层输入输出维度长度相同

Lec 10: Transformers (part 2) | 主题：Transformer Decoder→ masked self-attention→ output layer→ positional encoding→ residual/layerNorm

- d_{model} 表示空间/宽度，每个 token 的 hidden vector 都在同一个维度空间里，所以在 encoder 里输入 d_{model} ，输出也是 d_{model}
- Decoding in Transformer（Teacher Forcing）：一次只能生成一个 token，condition on 所有之前的生成好的 tokens，所以慢。因此有 teacher forcing 训练：在训练时有完整的 target sequence，直接让 decoder condition on ground-truth，这就是 TF。解决训练慢的问题。
- Masked self-attention：我们想处理整个 sequence in parallel。虽然把整句 target 都给了模型，但要模型在位置 t 预测时只能看见 1..t-1，而不能看见 t+1。因此做 attention 时候加一个 mask 把 future position 挡住。解决可以并行训练且不会看到未来。
- Output layer：功能是 converts decoder outputs into 概率分布 of vocabulary
- Positional encodings：self-attention 天生是不知道顺序的，所以要给每个位置 i 一个 position vector，把它和 token embedding 合成。
- 为稳定训练的选项：1. Residual：不直接学 $y=F(x)$ ，而学 $y=x+F(x)$ ，因为深度网络难以训练由于梯度爆炸/消失，残差链接提供一个 direct path for gradients，帮助 gradients flow。 2. Layer Normalization：将输出归一化到一个 consistent range，阻止输出 scale 过大。LayerNorm 独立应用在每个 token，在 hidden represent 上作 across features 的归一化（不是在 token 层面，每个 token 在自己的特征向量上作归一化）。
- Post-Norm：LayerNorm 在残差后使用，BERT 常用。Pre-Norm 是 layerNorm 在 sublayer 之前使用，这是目前 LLM 主流，可以 keeps residual path close to identity & better gradient flow stable training，阻止梯度消失

Lec 11: Pretraining: Encoders | 主题：表征学习 → ELM0→MLM（BERT）→ supervised fine-tuning → encoders vs decoders→KV caching

- 表征学习 representation learning：一个例子是用 DAN 做文本分类，他要学到的就是从 labeled data 里学到的参数 W 和 U。以后看到预训练+微调，就把预训练当作先学 W，微调就是换 U 或者调 W。W 更通用，U 更任务特化。
- Contextualized word representations: Word2Vec 方法给每个 word 分配一个 vector，导致 conflated meaning，这是静态词向量的根本缺陷。因此有 ELMo：用双向语言模型 Bi-LSTM LM 的中间 hidden states 来作为每个 tokens 的上下文表示。Freeze+特征提取
- BERT 与 MLM：和 ELMo 相比，用 transformer 代替 LSTM，用 bidirectional model with masked language modeling 替代标准 LM，fine-tune 而不是 freeze 在测试时候。ELMo 是 unidirectional 模型，BERT 是双向 bidirectional encoder 可以看到 past 和 future 的 tokens。所以为了能预测下一个词而不偷看，只能 Masked Language Modeling（MLM）。MLM 是 BERT 的两个预训练任务之一。
- Supervised Fine-tuning BERT on downstream：微调所有 parameters，用小学习率，小 epochs。那么 BERT 的全流程就是：输入 word piece tokens + position 给 transformer encoder →预训练随机 mask 一部分 tokens，在这些位置上作词表分类→微调全模型

- BERT 的强项是理解，匹配，分类，检索，不能生成 generate text。BERT 是 transformer encoder 模型，双向注意力集中机制+MLM，GPT 是 transformer decoder，单向注意力集中机制。BERT 的每个 hidden state 都依靠 all tokens
- 为什么 Decoder only 的比 BERT 更好？BERT 的生成需要 repeated recomputation，每一个新 token 都要计算整个 sequence representation。而 decoder 仅依靠 prefix，而 prefix 是过去的，一旦处理过就不会再改变
- 因此引出来 KV caching（在 autogressive inference 阶段），可以把过去的 attention 计算缓存 cache 起来，每步只算新增的 token，所以 Caching 只依靠 past 不靠未来，用新 query 去和缓存的 K/V 做 attention。所以 KV caching 是 decoder self-attention 的常见做法
- BERT 变种：1. RoBERTa：不改结构，只改规模和策略：160GB 而不是 16GB 2. ELECTRA：prediction 对每个 tokens 而不只是 masked tokens，因此同等算力下更多监督信号，学的更快 3. DeBERTa（最好）：把每个 word 拆分成 content vector 和 position vector

Lec 12: Pretrained Decoders | 主题：decoder-only → greedy/beam vs sampling/top-k/top-p/temperature → perplexity → encoder-decoder (BART/T5)

- Decoder LMs: A decoder LM uses a Transformer decoder with unidirectional (causal) self-attention: each token attends only to previous tokens. Decoder LM = causal self-attention + next token prediction。在 GPT 出来之前，主流的都是 BERT=预训练+监督微调，每种任务都需要 label 进行微调，但是 GPT 学会的是 continuation，在 zero-shot 下只靠 prompting。GPT=decoder only + causal LM
- Decoding 策略(inference decoding): 1. Greedy decoding: 每步选最大概率的 token，但容易 stuck in loop。2. Beam search: 找最 probable 的 full sequence，保留 top k 最可能的部分生成序列在每一步。3. Sampling: 随机从概率分布中取 tokens
- Beam search 是 max sequence probability，但是最大似然不等于高质量 text，这样可能导致 repetition 和 generic phrasing。他在 directed generation 例如翻译，摘要，数据到文本任务表现好，但是在 open-end generation 表现不佳
- 1. Pure Sampling: diversity ↑ 但 tail 风险 ↑ 有 long unreliable tail。2. Nucleus/Top-p Sampling: keeps the smallest set of tokens whose cumulative probability mass $\geq p$, truncating the unreliable tail。只保留累计概率至少为 p 的最小 tokens 集合 3. Top-k sampling: 每一步只保留概率最高的 k 个 token，重归一化后采样。
- Top-k 是固定词库 size，top-p 是固定概率 mass。Top-k 忽略分布形状，如果 flat 则小 k 丢失合理 token 导致太保守，如果 peaked 则大 k 纳入噪声导致不稳定。而 Top-p 比 top-k 更加 Adaptive，会根据分布自动改变保留的 token 数量。Top-p 在 entropy 随步骤变化时更好，有更多 human-like 多样性。Top-k 简单计算量便宜，低熵适合可预测领域 code 啥的
- 控制 Temperature 也可以控制 randomness: 高温高随机更 falt，多样性上升但更可能 incoherence；低温低随机更 peaked，更 coherence 但是会减少多样性。和 top-k 和 top-p 相比，温度不改变 token 而是控制概率分布。
- 评估 decoding 的办法是 Perplexity，beam search 经常得到更低 perplexity，但 更低 perplexity 不意味着更像人类文本。
- Encoder-Decoder Model: 真正的 seq2seq 任务（尤其高质量、受约束输出）在 encoder-decoder 上非常自然：encoder: 全上下文读输入，decoder: 左到右写输出，并 cross-attend 到输入表示。没有标注 (x,y) 怎么预训练 $P(y|x)$ ？从原始文本自己制造“合成输入-输出对”。要求：只用 raw text、自监督，并强迫 decoder 依赖 encoder 表示。
- BART 预训练：去噪重建，预训练 enc-dec 的经典方式：做破坏 corruption，例如 masking，删除，text infilling。BERT: 只有双向 encoder + MLM；不能自然做 seq2seq/生成。BART: encoder + autoregressive decoder；既能做 seq2seq，也可以只用 encoder 做 BERT 类任务。能不能做翻译/摘要在很大程度上由 有没有 decoder 决定。T5 是 text-to-text transformer，span corruption
- 为什么现代 LLM 都是 decoder only? 1. Unified objective: 从预训练到后训练到 prompting，都用 next-token prediction。2. Generality: 天然生成 (chat/code/story)，也能通过 prompting/fine-tune 做分类 3. Simplicity: 不需要 MLM 那种 masking

Lec 14: LLM Scaling Laws + Efficient Adaptation | 主题：scaling laws → Chinchilla-style → prompting/in-context → post-training → SFT/RLHF

- Scaling laws: 训练模型太贵，烧钱，碳排放巨大，所以要在小模型上做少量实验 → 拟合趋势 → 外推到大模型，而不是直接在大模型上瞎调。用小规模实验预测大规模表现，减少调参成本。Scaling laws 是把模型性能（比如 loss）表示为“规模”的函数：规模包括参数量 parameters N、数据量 D、计算量 compute。其目标是在给定 compute 下的最优可达 loss 边界 (optimal frontier)
- 一种办法是 Chinchilla / IsoFLOPs: 1. 固定一个目标 FLOPs budget; 2. 训练模型，改 N，调整训练 token 数使总 FLOPs 维持常数; 3. loss vs N 会呈“U 型”，用抛物线拟合并取最小点（最优 N）; 4. 对不同 FLOPs budget 重复 1-3; 5. 在 axes 上拟合 power law，实现对更大 FLOPs 的外推（最优 N、最优 D）。Scaling law 会 clean 和 robust，是经验规律，有前提：power law、均匀 token、固定模型
- Efficient adaptation: 微调贵要梯度更新，但 prompting 省成本且立即适配，可以 task adaptation at inference time。Prompting 本质：把任务表达为 next-token prediction。两种用 LM 做分类的方式：1. Generate and read the answer; 2. Compare label probabilities 更好
- Prompt Variability (为什么 prompting 玄学?)：同一任务不同 prompt，性能可大幅波动；prompt 的 perplexity 与效果相关，小改动可能大幅影响性能。Prompting 强但不鲁棒：对 wording 极度敏感。
- 1. Few-shot Prompting: 给例子在指示前，减少 ambiguity 但是 sensitive 对例子的 choice 和 ordering。2. Zero-shot prompting: 对 wording 高度敏感，prompt design 很大影响效果。3. CoT prompting: 让模型先写出来推理中间步骤再写结果。显式中间步骤相当于把复杂推理分解成多个局部可预测的 next-token 片段，减少“直接猜答案”的难度，提高准确率。4. self-consistency: 生成多条 CoTs 然后进行投票 majority vote。##好的 prompting 范式是减少歧义+提高鲁棒：1. 尝试描述成 next-token predict 2. Few-shot....
- Post-Training: 1. Instruction Tuning(SFT): 收集大量 instruction，在多任务上做监督微调，目标是最大化 token-level likelihood，缺点是只有 positive example 没有惩罚信号，收集高质量命令成本高昂，开放式任务没有标准答案 2. RLHF: Learn a reward model from human preference comparisons, optimize model to max reward. 因为 likelihood 不等于 preference，我们还 care 什么是坏的，RLHF 就可以学会惩罚不良行为

在 training 阶段使用 testset 的影响：data leakage, not trustworthy

n-gram LM, FFNN LM, RNN LM, Transformer, BoW 都会遇到 OOV 问题，只有 subword/BPE/ character-level 改变设计而不是改变 LM

属于监督学习的是：文本分类 (sentiment/spam)，序列标注 (RNN 可用于此)，机器翻译 MT in Lec8

属于无监督学习的是：Word2Vec/GloVe，所有 LM, ELMo, BERT, GPT, BART/T5

可以做分类任务的：BoW, DAN (FFNN 的一种)，LSTM (RNN 的优化)，Transformer

不可以做分类任务的：BPE/纯 Word2Vec/GloVe/单独的 attention layer/单纯 decoding 策略

Self-attention 不等于 muti-head attention，self attention 可以 single head 也可以 muti head。Transformer encoder = multi-head self-attention + FFNN; Transformer decoder = masked multi-head self-attention

在 decoding inference 里的温度是控制的 randomness，在 scale 点积里温度控制 concentration

随 sequence size T 增加的：attention weights, hidden states/embeddings, KV cache

随 vocabulary size V 增加的：embedding matrix, output projection/softmax layer

随 number of heads H 增加的：在不固定总模型维度 d_{model} 里，注意力/激活矩阵随 H 线性增长

1. Standard Seq2Seq LM 在长句子上表现如何? Why? This vanilla encoder-decoder model will perform poorly on long sentences because the decoder only gets to see the final encoder hidden state, which may not contain enough information about the input sequence. The model may struggle to accurately translate long sentences due to the limited context provided to the decoder. There is an information bottleneck.
2. 怎么让它在保持原来架构基础上解决长句子难题? We can use cross-attention, where the decoder attends to the encoder's hidden states at each time step. This modification allows the decoder to access more information about the input sequence, improving its ability to translate long sentences. 注意不能用 LSTM
3. Transformer 为什么可以处理 long-range dependencies 和 parallelization? 1. Long-range dependencies: Transformers achieve this through self-attention, which allows every token in a sequence to attend to all other tokens. This mechanism enables the model to directly capture relationships between distant tokens, as each token has global access to the entire sequence. 2. Parallelization: Self-attention supports parallel processing by handling all tokens in a sequence simultaneously. Unlike RNNs, which process tokens sequentially due to their reliance on recurrence, Transformers compute attention weights and outputs for the entire sequence in parallel, significantly speeding up training and inference.
4. 解释 Transformer 里的注意力权重的点积公式的 τ 的作用: The parameter τ controls the sharpness or spread of the attention distribution: 1. As $\tau \rightarrow 0$: The attention distribution becomes highly concentrated, with a small number of key vectors receiving most of the attention. This is because dividing $k \cdot v$ by a small τ exaggerates differences in dot products. The exponentials corresponding to the largest dot products dominate, leading to nearly one-hot attention weights. 2. As $\tau \rightarrow \infty$: The attention distribution becomes increasingly uniform, with all key vectors receiving similar attention. Dividing $k \cdot v$ by a large τ reduces differences between dot products, causing all exponentials to approach 1. The normalization then assigns nearly equal weight to all keys. The temperature τ adjusts the focus of the attention mechanism: smaller τ sharpens the focus on a few keys, while larger τ spreads the focus evenly across all keys.
5. 在训练中, 观察到训练损失剧烈震荡, 有时甚至在训练步骤之间急剧增加, 最可能的原因是什么? 你会做出怎样的调整? The learning rate is likely too large. With a large step size, the gradient updates overshoot the minimum, causing the loss to increase or oscillate between steps. Reducing the learning rate would help stabilize training by allowing smaller updates that are less likely to diverge.
6. 用 Beam Search 进行 text 生成。在哪些类型的任务中, 它可能不如基于采样的方法有效? 请举例说明并解释原因。Beam search may not be preferred in open-ended tasks such as story writing or dialogue generation. Because it maximizes sequence-level likelihood, it often produces generic or repetitive high-probability outputs. In open-ended tasks, many continuations are valid, and sampling allows greater diversity. In contrast, for constrained tasks like machine translation or summarization, where correct limited, beam search is more appropriate since likelihood better aligns with correctness.
7. 为啥在 residual connection 之前用 normalization (Pre-Norm) 可以稳定深度 Transformer 的训练? “In very deep transformers, gradients can vanish or explode as they pass through many nonlinear layers. In Pre-Norm, normalization is applied before the transformation $F(\cdot)$, so the block becomes $y = x + F(\text{LN}(x))$ instead of post-Norm $y = \text{LN}(x + F(x))$. This keeps the residual (identity) path x unchanged, there is always a direct path for gradients to flow back through the identity connection, which helps prevent vanishing gradients even if $F(\cdot)$ causes gradients to vanish.
8. 在 post-training, 为什么使用监督式微调 (SFT) maximizing likelihood 并不一定能 max human preference? Supervised fine-tuning maximizes token-level likelihood of the reference responses in the dataset. These objective rewards matching the exact tokens in demonstrations. However, many prompts have multiple valid responses, and human preference depends on sequence-level qualities such as helpfulness, harmlessness, and honesty. As a result, a response can have high likelihood under the training data but still be less preferred by humans for being unhelpful, unsafe, or dishonest
9. 为什么增加 vocabulary size 会增加 softmax 层的计算成本? Increasing vocabulary size raises softmax cost because the model must score every token each step. The output projection has shape $d_{\text{model}} \times \text{vocab_size}$, so producing logits requires vocab_size dot products between the hidden vector and the matrix columns, plus softmax normalization over vocab_size entries. In training this happens for every position in batch example ($\text{batch_size} \times \text{sequence_length} \times \text{vocab_size}$), and in inference it happens once per generated token, so larger vocabularies make each decoding step slower.
10. 为啥在 attention 中要按 $\sqrt{d_k}$ 进行缩放? 为什么是 $\sqrt{d_k}$ 而不是除以 d_k ? We scale dot-product attention by the square root of d_k to keep the attention scores at a reasonable magnitude as d_k grows. Without scaling, dot products tend to get larger in absolute value in higher dimensions, which makes the softmax distribution become extremely peaked (almost one-hot). When softmax saturates like that, most probabilities go to near zero, gradients become tiny, and training becomes unstable or slow because the model can't adjust attention weights effectively.
11. 为什么 chain-of-thought prompting 未必能提升 post-trained models 的性能? Chain-of-thought prompting may not improve post-trained models because post-training (instruction tuning and often preference optimization like RLHF) already shapes the model to follow instructions, be helpful, and produce step-by-step explanations when they are appropriate. In other words, the behavior that CoT prompting tries to elicit is often “baked in” by the post-training data and objectives, so adding an explicit “think step by step” cue can be redundant and yield little gain. Also, the post-training objective optimizes for user-preferred outputs (clarity, safety, usefulness) rather than purely maximizing likelihood of the next token under raw web text. Because the model is already trained to choose a good response style, extra CoT prompting may not change the internal computation much, or it may even hurt by pushing the model to generate longer reasoning than needed, increasing chances of errors or verbosity.
12. KV caching 在 autoregressive inference 的目的是啥? KV caching speeds up autoregressive inference by avoiding repeated computation on the already-generated prefix. Without caching, each new token would require recomputing attention keys and values for all previous tokens in every decoder layer, making generation much slower as the sequence grows. With KV caching, we store the past keys and values once per layer and reuse them; at each step we only compute the new token's key, value, and query, then attend to the cached past. This reduces per-step work to “new token plus attention over the prefix,” so generation scales much more efficiently with sequence length.
13. 如果温度 $T \rightarrow 0$, what decoding strategy does sampling approximate? As temperature T goes to 0, sampling approximates greedy decoding (argmax). The softmax distribution becomes extremely peaked: the highest logit dominates, its probability goes to nearly 1, and the probabilities of all other tokens go to nearly 0. As a result, sampling almost always picks the single highest-scoring token at each step.